

Working Set Page Replacement Algorithm

1 Core Idea

Definition

The **working set** of a process is the set of pages the process is currently using.

The working set algorithm tries to keep the current working set in memory. When a page fault occurs, it prefers to evict a page that is **not** in the working set.

Replacement Goal

evict a page not in the current working set

2 Demand Paging

Demand Paging

Demand paging loads a page only when the process actually references it.

At process startup, none of the process's pages may be in RAM. The first instruction fetch causes a page fault. More page faults usually follow for global data, heap pages, and stack pages.

After a short startup period, the process normally has the pages it needs and page faults become less frequent.

3 Locality of Reference

Most programs do not use their whole address space uniformly.

Locality

During one phase of execution, a process usually references a small cluster of pages repeatedly.

Example: one pass of a compiler may use parser code and symbol-table pages. A later pass may use optimization code and different data pages. The working set changes, but usually slowly.

4 Thrashing

Thrashing

A process is **thrashing** when it causes page faults so often that most time is spent handling faults instead of executing useful instructions.

Thrashing happens when available memory is too small to hold the process's working set.

Why Thrashing Is Bad

A normal instruction may take nanoseconds, but reading a page from disk may take milliseconds. If page faults occur every few instructions, the program becomes extremely slow.

5 Working Set Model

Let:

- k = number of most recent memory references considered;
- t = current time;
- $w(k, t)$ = set of pages used by the k most recent memory references at time t .

Working Set

$$w(k, t) = \{\text{pages referenced in the } k \text{ most recent memory references at time } t\}$$

As k grows, the working set can only stay the same or grow. It cannot shrink, because looking farther back can include more pages.

Monotonic Behavior

$$k_1 < k_2 \quad \Rightarrow \quad w(k_1, t) \subseteq w(k_2, t)$$

6 Prepaging

Prepaging

Prepaging loads a process's expected working-set pages before allowing the process to run.

When a process is swapped out and later resumed, the OS can use its previous working set as a guess about which pages it will need again.

Purpose

Prepaging reduces the burst of page faults that would otherwise happen when a process restarts.

7 Time-Based Working Set

Counting the last k memory references exactly is too expensive. Real systems often approximate the working set using process execution time.

Virtual Time

Virtual time is the amount of CPU time a process has actually used.

Instead of using the last k references, define the working set as pages referenced during the last τ seconds of virtual time.

Time-Based Working Set

working set = {pages referenced during the last τ seconds of virtual time}

8 Working Set Algorithm Figure

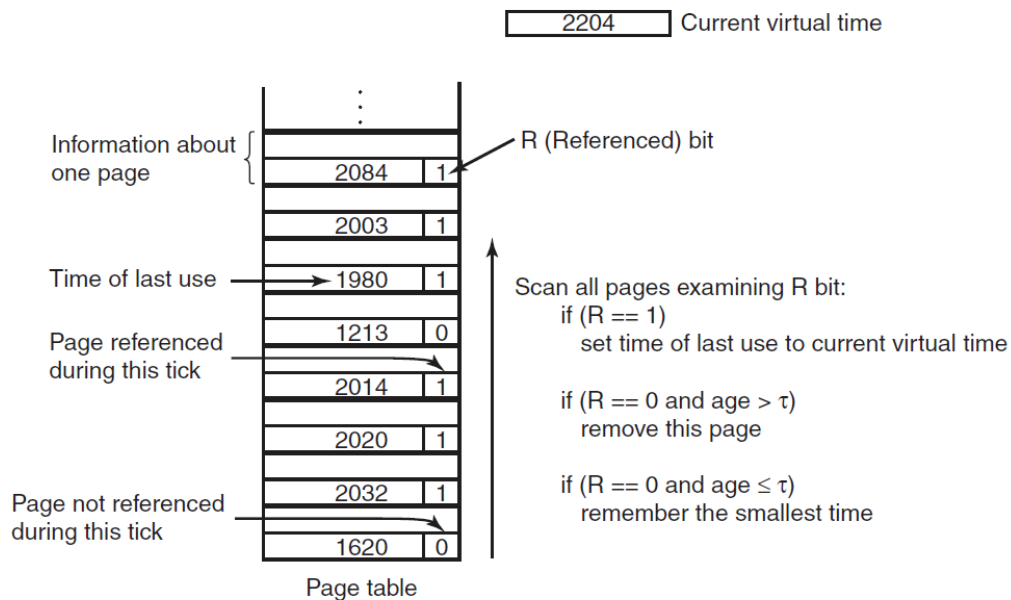


Figure 3-19. The working set algorithm.

What the Figure Shows

The page table stores each page's approximate time of last use and its R bit. Current virtual time is 2204.

9 How to Read the Figure

Field	Meaning
Current virtual time 2204	CPU time already used by this process.
Time of last use R bit	Approximate virtual time when this page was last referenced. 1 means the page was referenced during the current clock tick; 0 means it was not.
Age	Current virtual time minus time of last use.

Page Age

$$\text{age} = \text{current virtual time} - \text{time of last use}$$

10 Algorithm Cases

On a page fault, the OS scans page-table entries for pages that are currently in memory.

Case	Action
$R = 1$	Page was referenced during this tick. Set its time of last use to the current virtual time. Do not evict it.
$R = 0$ and $\text{age} > \tau$	Page is outside the working set. Evict it.
$R = 0$ and $\text{age} \leq \tau$	Page is still inside the working set. Remember it only as a fallback candidate.

11 Worked Example from the Figure

Assume:

$$\text{current virtual time} = 2204, \quad \tau = 150$$

Consider the page with:

$$\text{time of last use} = 1980, \quad R = 1$$

Since $R = 1$, the page was referenced during this clock tick. The OS updates its time of last use:

$$1980 \rightarrow 2204$$

It is kept in memory.

Referenced Page

If $R = 1$, the page is definitely in the working set, so it is not a victim.

Now consider the page with:

$$\text{time of last use} = 1620, \quad R = 0$$

Compute its age:

$$\text{age} = 2204 - 1620 = 584$$

Compare with $\tau = 150$:

$$584 > 150$$

Eviction Candidate

This page has not been used recently enough, so it is outside the working set and can be evicted.

12 If No Page Is Clearly Outside

Sometimes the scan finds no page with:

$$R = 0 \quad \text{and} \quad \text{age} > \tau$$

Then all pages appear to be in the working set.

Situation	Fallback action
Some pages have $R = 0$	Evict the $R = 0$ page with greatest age.
All pages have $R = 1$	Choose a page randomly, preferably a clean page.

13 Why the Exact Definition Is Too Expensive

The exact definition uses the last k memory references. A direct implementation would need to record every memory reference in a large shift register.

Too Expensive

Maintaining and processing the exact last k memory references would be prohibitively expensive, so practical systems use approximations.

14 Strengths and Weaknesses

Point	Explanation
Strength	Reduces page faults by keeping active pages in memory.
Strength	Helps avoid thrashing.
Strength	Supports prepaging when a process resumes.
Weakness	Needs bookkeeping for last-use time and R bits.
Weakness	Choosing τ well is important.

15 Final Summary

One-Box Summary

A working set is the group of pages a process is actively using.
If the working set fits in RAM, page faults stay low.
If it does not fit, the process may thrash.
The algorithm evicts pages not used within the last τ virtual-time units.